

Speech Command Classification

Differentiating Spoken Commands with a CNN on Mel-Spectrograms

Hadi Afyouni

ESIGELEC Univ

Deep Learning Project - 2026

1. Objective

- Build a deep-learning system that differentiates spoken commands
- Input: a 1-second audio clip from the microphone
- Output: one of 10 command words
 - yes, no, up, down, left, right, on, off, stop, go
- Trained on a public dataset, then runs LIVE from the mic
- Same CNN approach as the image labs - applied to audio

Commands

- | | |
|--------|---------|
| - yes | - no |
| - up | - down |
| - left | - right |
| - on | - off |
| - stop | - go |

- Google Speech Commands dataset (v0.01), via Kaggle
 - Same family as the TensorFlow speech_commands catalog
- 64,727 WAV files across 30 word folders
- Each clip: mono, 1 second, 16 kHz
- ~2,370 clips per target word (10 words used)
- Includes a _background_noise_ folder (used for augmentation)

3. Method - Processing Pipeline

Raw audio is turned into an image (spectrogram), then classified by a CNN:

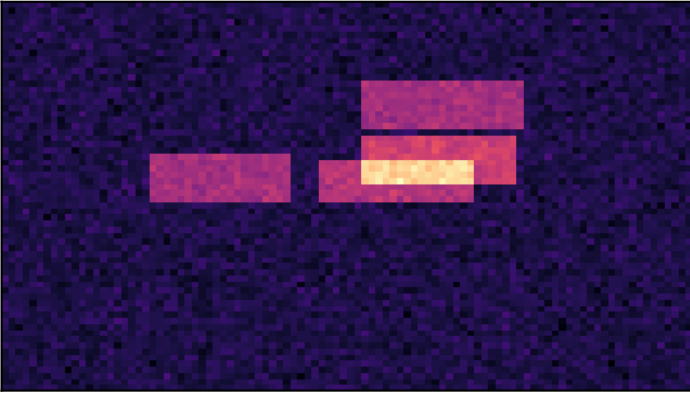


- STFT: 25 ms window, 10 ms hop, 512-point FFT
- Mel filterbank: 64 bands over 80-7600 Hz (speech range)
- Output spectrogram: $\sim 98 \times 64$ (time x frequency)

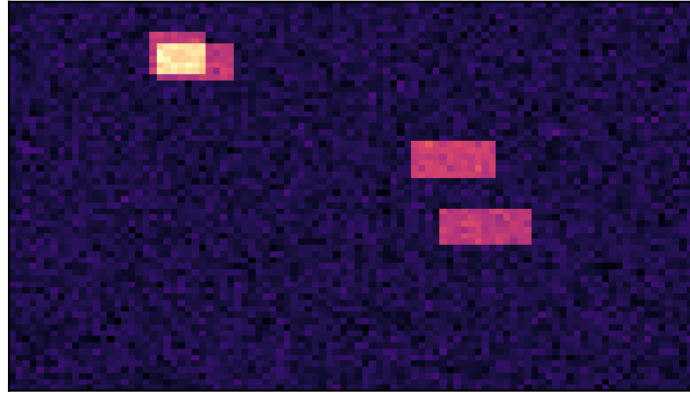
3.1 Feature: Log-Mel Spectrogram

Each word produces a distinct time-frequency pattern the CNN learns to recognise:

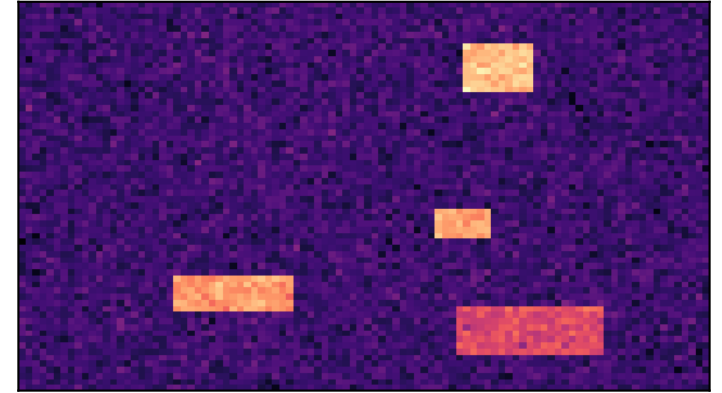
yes



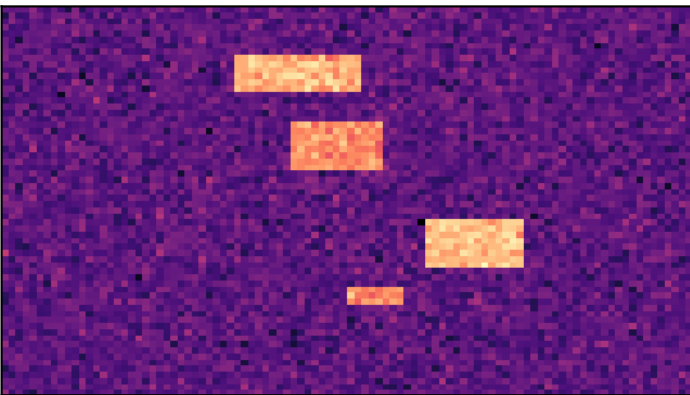
no



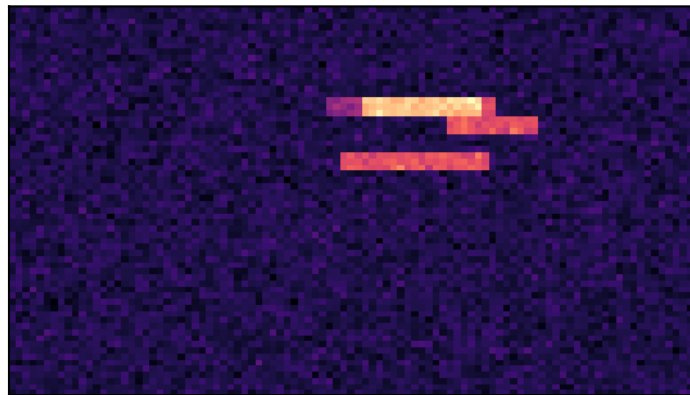
up



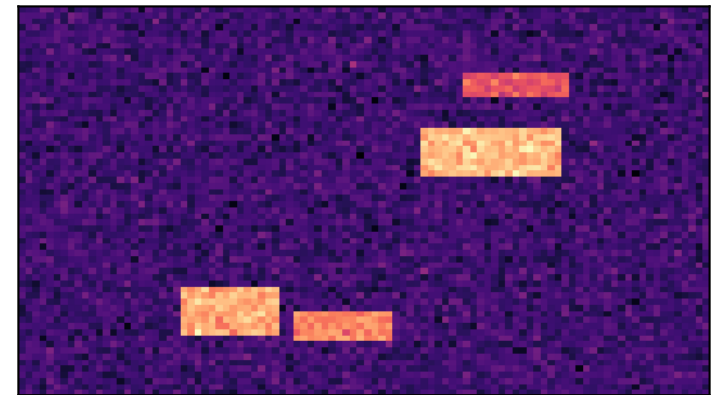
down



left



right



(illustrative spectrogram shapes)

3.2 Data Augmentation

Every clip is loaded twice (clean + augmented) -> dataset doubled to ~47,000 samples

Time shift

Word shifted +/- 100 ms
in time

Gaussian noise

Simulates mic and
electrical noise

Background mix

Real noise clips mixed
in (30% chance)

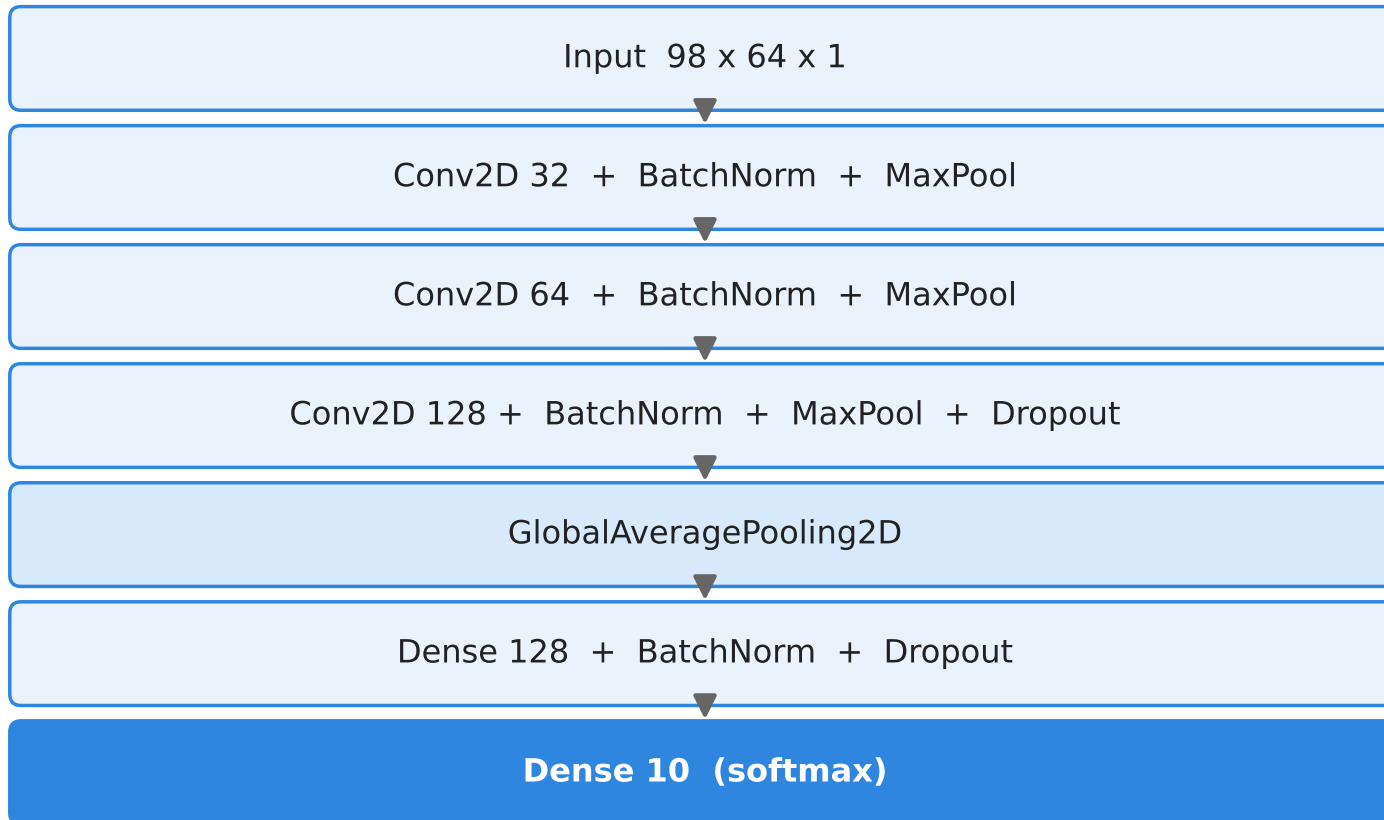
SpecAugment

Mask random frequency
bands and time steps

Per-sample normalisation: each spectrogram is normalised by its own mean/std

-> makes the model robust to recording volume and microphone differences

4. CNN Architecture



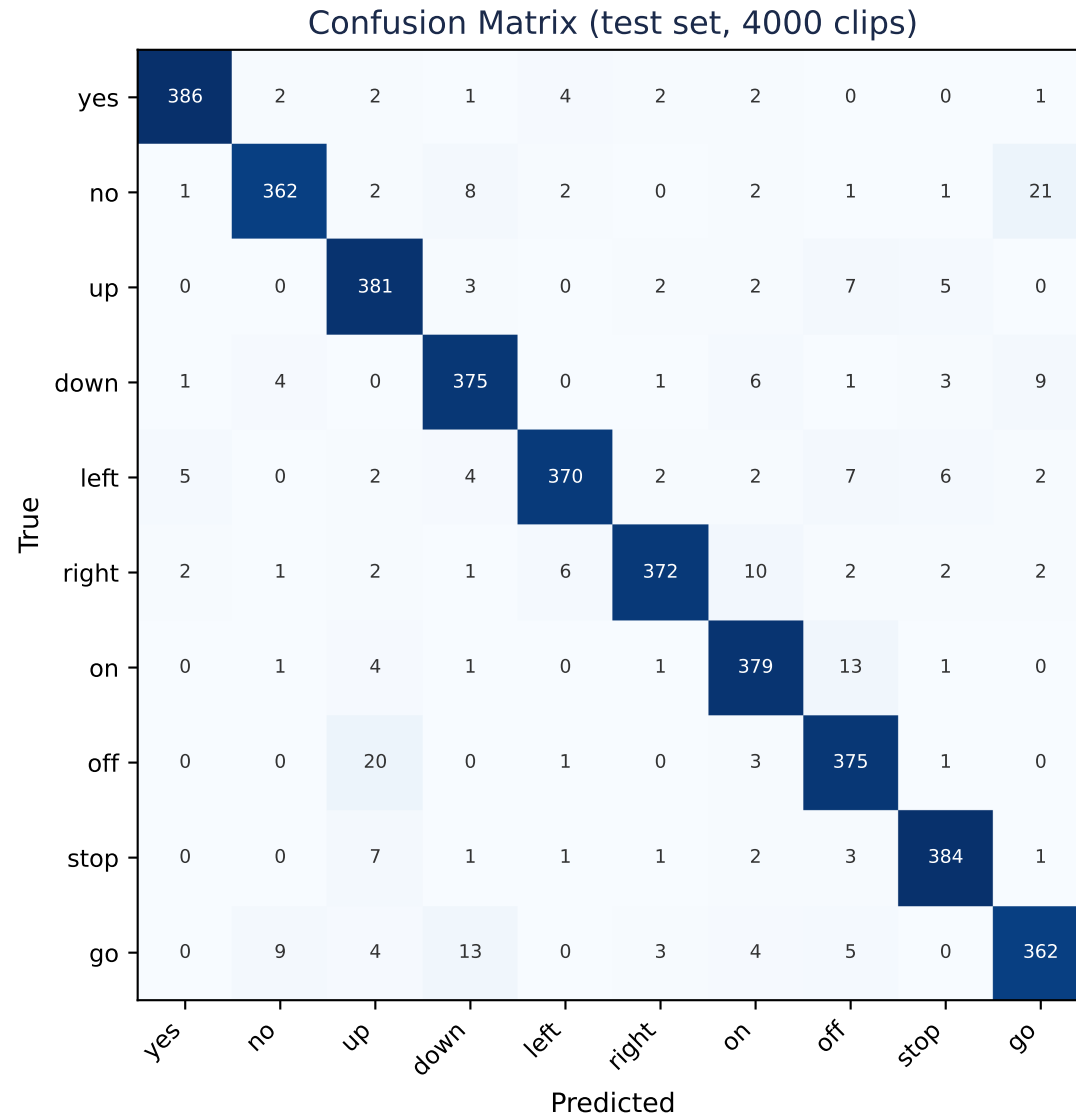
- BatchNorm:
 - stabilises training,
 - helps generalise
- GlobalAvgPool:
 - fewer params,
 - less overfitting
- Dropout:
 - regularisation

5. Training Configuration

Optimizer	Adam
Loss	Sparse categorical cross-entropy
Batch size	64
Max epochs	30 (EarlyStopping usually stops earlier)
Data split	80% train / 20% test (stratified)
ReduceLROnPlateau	halve learning rate when val-loss stalls
EarlyStopping	stop on plateau, restore best weights

Hardware: TensorFlow on Windows runs CPU-only (no GPU). Tuned to ~3.7 min/epoch, ~1 hour total.

6. Results



93.7%

test accuracy

- Strong diagonal = correct predictions
- Errors only in similar-sounding pairs:
 - no $\leftrightarrow$ go
 - off $\rightarrow$ up
- Linguistically reasonable confusions,
- not random errors

7. Live Microphone Inference

1

Record 1 second from the microphone

2

Auto-detect mic sample rate (e.g. 48 kHz) and resample to 16 kHz

3

Compute log-mel spectrogram + per-sample normalisation

4

CNN predicts the command + confidence %

5

If confidence < 75% -> print "I don't know" instead of guessing

8. Challenges & Solutions

Challenge

Solution

Python 3.14 had no TensorFlow wheel

-> Installed Python 3.12, rebuilt the venv

TF download dropped on 4G

-> Resume-capable download + faster uv installer

GPU not usable by TF on Windows

-> Tuned model to fit a CPU training budget

Live mic confused "left" with "up"

-> Fixed sample-rate + per-sample normalisation

Over-confident wrong guesses

-> Added 75% confidence threshold ("I don't know")

Key lesson: the bug was a train/inference MISMATCH, not model capacity. Fixing the pipeline solved it.

9. Conclusion

- A working speech-command classifier:
 - Trains in ~1 hour on CPU to ~93-94% test accuracy
 - Generalises to live mic input via correct audio preprocessing
 - Rejects uncertain inputs instead of guessing
- Demonstrates the full deep-learning workflow on audio:
 - data prep -> augmentation -> CNN -> training -> live deployment
- Same CNN family as the image labs, applied to sound

Differentiating speech commands with deep learning - done.